Scaled Dot-Product Attention: Why Divide by the Square Root of d_k

Contents

1. Overview 2
2. Prerequisites 3
3. What d_k is 3
4. Longer vectors give more spread-out dot products 5
   4.1 The variance grows exactly like d 6
   4.2 Why high-dimensional vectors are used anyway 7
5. Why a wide spread is a problem: the softmax saturates 7
6. Choosing the scaling factor 11
7. The full formula in code 13
8. Summary 15
9. Sources 15
10. Key terms 16

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Self-attention (query, key, value).
- **Leads to:** Multi-head attention.

1. Overview

Key point: The transformer divides every **attention score** (G-223) by $\sqrt{d_k}$, the square root of the length of the key vectors, before the softmax (the function that turns scores into weights that add up to 1). Without that step, long vectors give scores so spread out that one word takes almost all the weight and almost no gradient (the signal that tells each weight how to change) flows back.

Dividing by $\sqrt{d_k}$ brings the **variance** (G-2074; how spread out the scores are) back to about 1, whatever the length.

The idea of a word as a mix of its sentence built self-attention from first principles: every word gets a query, a key and a value vector. Stacking the vectors of all the words as rows gives three matrices (tables of numbers): Q (queries), K (keys) and V (values), one row per word. The contextual embeddings are $\text{softmax}(QK^T)\,V$. The paper that introduced the transformer, "Attention Is All You Need", uses the same formula with one extra step (Vaswani et al. 2017, eq. 1):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Because of that division, the paper calls its attention **scaled dot-product attention** (G-1745). Figure 1 is a flow chart read from top to bottom: it shows where the new step sits. This Note answers two questions: what d_k is, and why the scores must be divided by exactly $\sqrt{d_k}$.

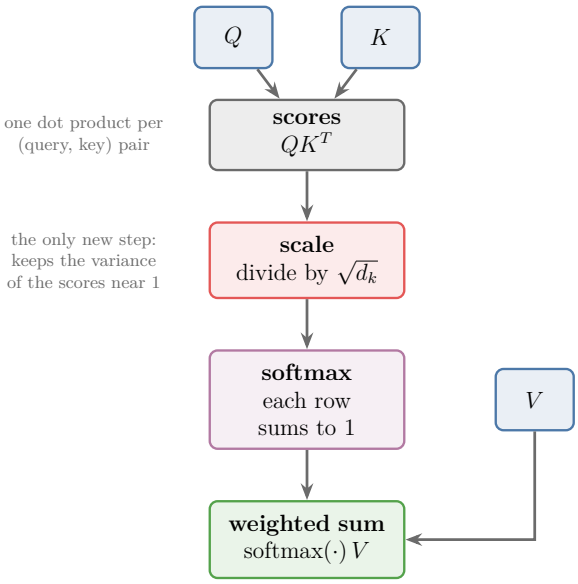


Figure 1: Scaled dot-product attention: the scores QK^T are divided by $\sqrt{d_k}$ before the softmax. Everything else is self-attention without scaling

2. Prerequisites

- Query, key and value vectors, and softmax$(QK^T)\,V$.
- The softmax function: it turns a list of scores into weights that sum to 1.
- **Variance of a random variable** (G-2076): the average squared distance from the mean.
- The vanishing gradient problem: why tiny gradients stop a network from learning.

3. What d_k is

> **Key point:** d_k is the number of values in each key vector. Every score in QK^T is divided by $\sqrt{d_k}$ before the softmax.

Take the sentence "money bank grows". Each word has an embedding, and multiplying the embedding by the three learned matrices W_Q, W_K and W_V gives its query, key and value vectors. With embeddings of 3 numbers and 3×3 matrices, every key vector has 3 numbers, so $d_k = 3$. With 512-number keys, $d_k = 512$. Queries and keys must have the same length, since we take their dot products; in the transformer paper $d_k = 64$ per attention head (Vaswani et al. 2017, section 3.2.2).

The matrix QK^T (K^T is K with its rows and columns swapped, so that each row of Q meets each key) holds one dot product (multiply two vectors entry by entry and add) for every pair of a query and a key: for 3 words, 9 scores:

$$3 \times 3 = 9$$

Scaling divides each of the 9 scores by $\sqrt{d_k}$; the softmax then works on the scaled scores, row by row.

1. **In words:** compute the query's dot product with every key, divide each by $\sqrt{d_k}$, then apply the softmax.
2. **Formula:** for query q and keys $k_1, \ldots, k_n$, the weights are

$$\alpha_j = \text{softmax}_j\left(\frac{q \cdot k_j}{\sqrt{d_k}}\right)$$

3. **Example:** the query of "bank" is $q = (1, 2, 1)$; the keys of "money", "bank", "grows" are $(2, 1, 0)$, $(1, 2, 1)$ and $(0, 1, 0)$. The three scores are the dot products:

$$1 \times 2 + 2 \times 1 + 1 \times 0 = 4$$
$$1 \times 1 + 2 \times 2 + 1 \times 1 = 6$$
$$1 \times 0 + 2 \times 1 + 1 \times 0 = 2$$

- Unscaled:

$$\text{softmax}(4, 6, 2)$$

$$= (0.117,\ 0.867,\ 0.016)$$

- Scaled: divide the scores by $\sqrt{3}$.

$$\sqrt{3} = 1.732$$

$$\frac{4}{1.732} = 2.31$$

$$\frac{6}{1.732} = 3.46$$

$$\frac{2}{1.732} = 1.15$$

$$\text{softmax}(2.31,\ 3.46,\ 1.15)$$

$$= (0.223,\ 0.707,\ 0.070)$$

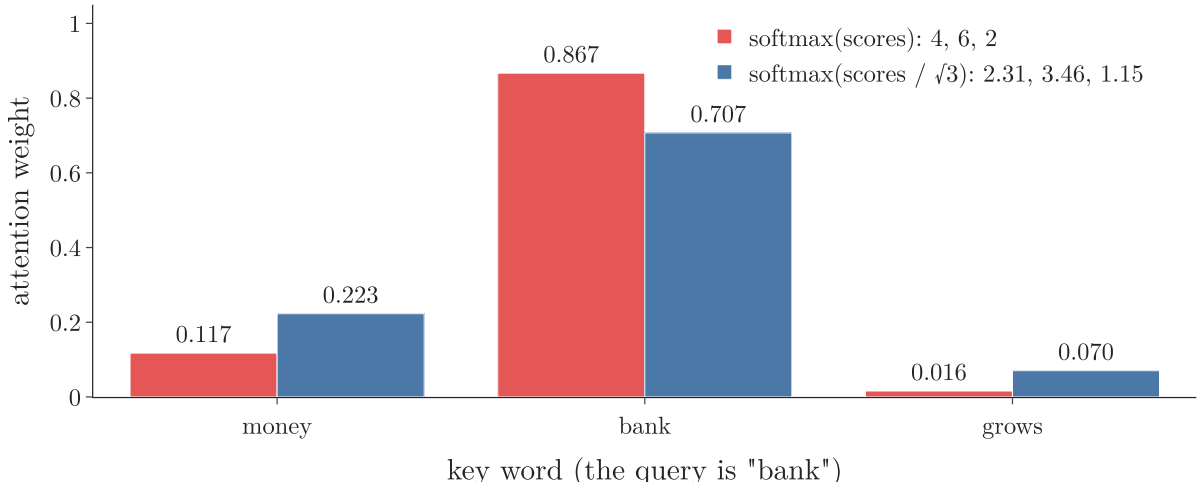


Figure 2: The worked example. Red: the softmax of the raw scores 4, 6, 2. Blue: the softmax after dividing by $\sqrt{3}$. The scaled weights are less extreme

The scaled weights are less extreme (Figure 2): "bank" still attends most to itself, but "money" now gets 22% of the weight instead of 12%. The rest of this Note shows why that matters more and more as d_k grows.

4. Longer vectors give more spread-out dot products

> **Key point:** The **variance** (G-2074) of a dot product grows in proportion to the length of the vectors. Dot products of 1,000-number vectors have about 350 times the variance of those of 3-number vectors.

Each score in QK^T, an **attention score** (G-223), is a dot product of two vectors. Take 1,000 pairs of random vectors, every number drawn independently with mean 0 and variance 1, and compute the 1,000 dot products. Then repeat with longer vectors (Notebook):

Length d	Range of the 1,000 dot products	Variance
3	−7.2 to 5.8	2.9
100	−27.6 to 32.4	96.7
1,000	−105.7 to 95.8	1,015.6

Figure 3 is a set of histograms: the horizontal axis is the value of a dot product, and the height of each bar counts how many of the 1,000 dot products fall in that range. A wide histogram means widely spread values.

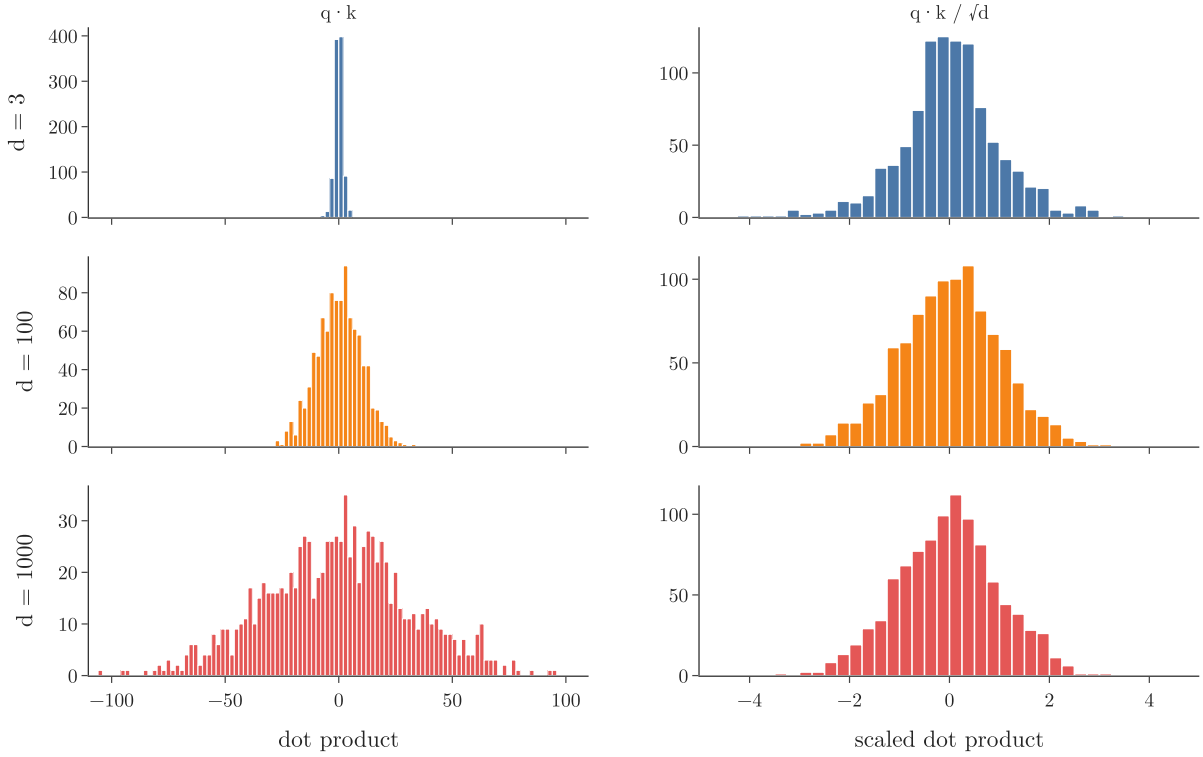


Figure 3: Histograms of 1,000 dot products each, one row per vector length d (each row has its own count scale). Left: the spread grows with d. Right: after dividing by $\sqrt{d}$, the three histograms have the same spread

The variance is close to d every time (Figure 3, left). The reason is simple: a dot product of length d is a sum of d products, and every extra term adds its own variation:

$$q \cdot k = q_1 k_1 + q_2 k_2 + \cdots + q_d k_d$$

A sum of 3 random terms stays small; a sum of 1,000 random terms can wander far from 0.

4.1 The variance grows exactly like d

> **Key point:** If the numbers in q and k are independent with mean 0 and **variance** (G-2074) 1, then $q \cdot k$ has mean 0 and variance exactly d_k (Vaswani et al. 2017, footnote 4).

1. **In words:** each product $q_i k_i$ has variance 1, the d products are independent, and variances of independent terms add up.
2. **Formula:** here $E[x]$ means the average value of x and $\mathrm{Var}(x)$ its variance. With $E[q_i] = E[k_i] = 0$ and $\mathrm{Var}(q_i) = \mathrm{Var}(k_i) = 1$, all independent,

$$\mathrm{Var}(q_i k_i) = E[q_i^2 k_i^2] - (E[q_i k_i])^2$$

For independent numbers:

$$E[q_i^2 k_i^2] = E[q_i^2]\, E[k_i^2]$$

$$E[q_i^2 k_i^2] = 1 \cdot 1 = 1$$

$$E[q_i k_i] = E[q_i]\, E[k_i]$$

$$E[q_i k_i] = 0 \cdot 0 = 0$$

$$\mathrm{Var}(q_i k_i) = 1 - 0^2 = 1$$

$$\mathrm{Var}(q \cdot k) = \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) = d_k$$

3. **Example:** for $d = 1$, 2 and 3, the variance is 1, 2 and 3: a vector one number longer adds one more unit of variance. The Notebook measures 0.99, 1.96, 4.00, 7.99, … and 1,048 for $d = 1, 2, 4, 8, \ldots, 1{,}024$, with 20,000 pairs each (Figure 4, left).

4.2 Why high-dimensional vectors are used anyway

> **Key point:** Short vectors would keep the **variance** (G-2074) low, but they hold too little information about each word. We keep long vectors and fix the variance instead.

One way to avoid the problem would be short embeddings. But a vector of a few numbers cannot capture much about a word's meaning; the transformer paper uses embeddings of 512 numbers (Vaswani et al. 2017, section 3.1). So the vectors stay long, and the variance of the scores must be brought down some other way.

5. Why a wide spread is a problem: the softmax saturates

> **Key point:** When the scores spread widely, the softmax puts nearly all the weight on the largest score. Its gradient then becomes almost 0 for every score, and the attention weights hardly learn.

The softmax exponentiates its inputs, so it reacts to differences between scores. Close scores give comparable weights; scores far apart push one weight towards 1 and the others towards 0. A softmax pushed to these extremes is said to be **saturated** (G-1740). In the example of section 3, the scores $(4, 6, 2)$ already gave "bank" 87% of the weight; with scores ten times larger, $(40, 60, 20)$, it would get more than 99.9999%. Goodfellow et al. (2016, section 6.2.2.3) describe exactly this: the softmax saturates "when the differences between input values become extreme".

A saturated softmax harms training. Gradient descent changes the attention weights through the gradient of the softmax, the **softmax gradient** (G-1831). The symbol $\partial \alpha_i / \partial s_j$ means how much the weight α_i changes when only the score s_j changes a little. Write $S = \sum_m e^{s_m}$, so that $\alpha_i = e^{s_i}/S$. Take the weight α_i and differentiate it with respect to its own score s_i, one line per step (quotient rule):

$$\begin{aligned}\frac{\partial \alpha_i}{\partial s_i} &= \frac{e^{s_i}\, S - e^{s_i}\, e^{s_i}}{S^2} \\ &= \frac{e^{s_i}}{S} \cdot \frac{S - e^{s_i}}{S} \\ &= \alpha_i (1 - \alpha_i)\end{aligned}$$

Now differentiate α_i with respect to a different score s_j ($j \neq i$), where e^{s_i} does not change and only S does:

$$\begin{aligned}\frac{\partial \alpha_i}{\partial s_j} &= \frac{0 \cdot S - e^{s_i}\, e^{s_j}}{S^2} \\ &= -\frac{e^{s_i}}{S} \cdot \frac{e^{s_j}}{S} \\ &= -\alpha_i \alpha_j\end{aligned}$$

Together, these derivatives for every pair of a weight α_i and a score s_j fill a table, one row per weight and one column per score: the **gradient matrix** of the softmax. Its diagonal holds $\alpha_i(1 - \alpha_i)$, every other entry $-\alpha_i \alpha_j$.

Check on the scores $(4, 6, 2)$ of section 3, where "bank" has $\alpha = 0.867$:

$$\alpha(1 - \alpha) = 0.867 \times 0.133 \approx 0.115$$

If one weight is close to 1 and the rest close to 0, every entry is close to 0: for the big weight $\alpha_i(1 - \alpha_i) \approx 1 \times 0$, for the small ones $\alpha_i \approx 0$. Little gradient flows back to the scores, and so to W_Q and W_K: the same **vanishing gradient** (G-2070) problem as in deep networks. Vaswani et al. (2017, section 3.2.1) give this as their reason for scaling: for large d_k the dot products "grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients".

An analogy: a teacher asks a class to raise hands with their questions. If a few children are much taller than the rest, only their hands are seen, and only their questions get answered; the others never take part in the lesson. If all the children are about the same height, the teacher sees every hand. Scaling makes all the scores "about the same height", so every word takes part in training.

The Notebook measures the saturation directly: one random query against 10 random keys, 2,000 times for each length d. For each softmax it records the largest weight and the size of the gradient matrix above: the square root of the sum of its squared entries, one number that is near 0 only when every entry is near 0 (Figure 4, middle and right). In Figure 4 the horizontal axes, and the vertical axis of the left panel, are logarithmic: each labelled gridline (1, 10, 100, 1000) is ten times the one before.

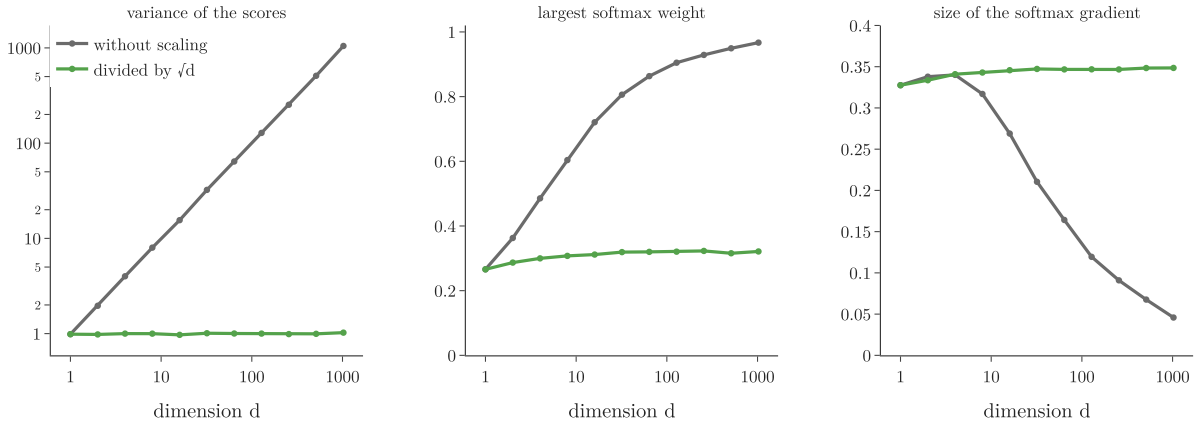


Figure 4: Left: the variance of the scores grows like d without scaling and stays at 1 with it (log scales). Middle: the mean largest weight of a 10-word softmax. Right: the mean size of the softmax gradient. Without scaling, the softmax saturates as d grows and its gradient shrinks; with scaling, both stay flat

d	Largest weight, unscaled	Largest weight, scaled	Gradient size, unscaled	Gradient size, scaled
1	0.27	0.27	0.33	0.33
64	0.86	0.32	0.16	0.35
512	0.95	0.32	0.068	0.35
1,024	0.97	0.32	0.046	0.35

Without scaling, at $d = 1{,}024$ one word takes 97% of the weight on average, and the gradient is 7.6 times smaller than with scaling. With scaling, both numbers stay the same from $d = 64$ on (0.32 and 0.35). At $d = 1$ they are a little lower, because with one number per vector all 10 scores are that single query number times a key number: when the query number happens to be near 0, every score is near 0 and the weights come out nearly even.

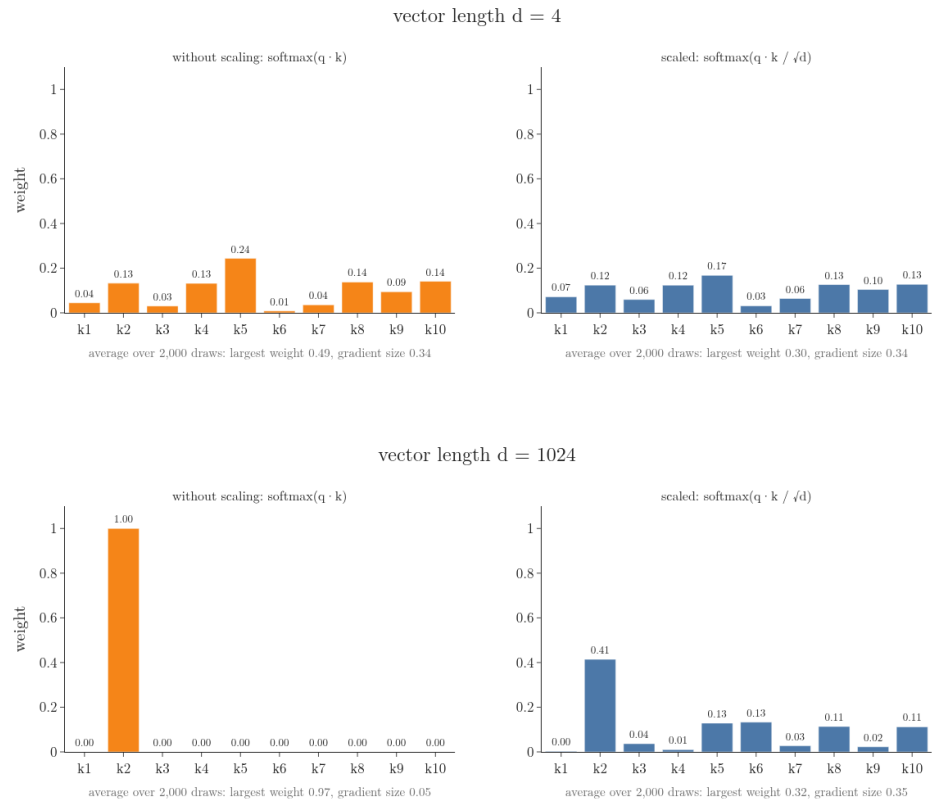


Figure 5: One random query against 10 random keys, using the first d numbers of each, for d from 1 to 1,024. Left: the softmax of the raw dot products collapses onto one key. Right: divided by $\sqrt{d}$, the weights stay spread out. Under each panel, the Notebook's averages over 2,000 draws

Figure 5 shows the same thing happening to one softmax as d grows. Watch the left panel: as the vectors get longer, one bar grows towards 1 and the others shrink to 0. The right panel, with the same query and keys, stays spread out: at $d = 1{,}024$ its largest bar is 0.41, against 1.00 on the left.

The same effect shows on a real sentence. Figure 6 is a grid for each case (rows are the words as queries, columns the words they look at, and a darker cell is a larger weight). It takes the first 10 words of the first IMDB training review, with random 512-number embeddings and random W_Q, W_K, as at the very start of training. Without scaling, 6 of the 10 rows put more than 99% of their weight on a single word. With scaling, no row does, and every word receives some attention.

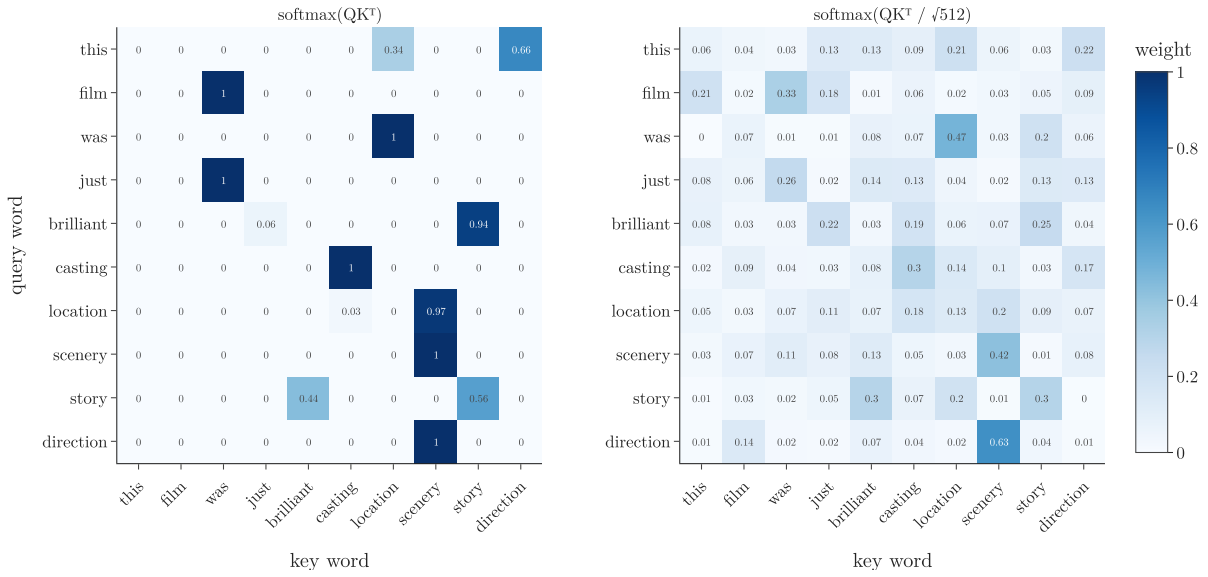


Figure 6: Attention weights for "this film was just brilliant casting location scenery story direction", $d_k = 512$, untrained random weights. Left: without scaling, most rows are a single 1. Right: divided by $\sqrt{512}$, the weights are spread out

6. Choosing the scaling factor

> **Key point:** Multiplying a random variable by c multiplies its **variance** (G-2074) by c^2. Since the variance of the scores is d_k, dividing by $\sqrt{d_k}$ brings it back to $d_k/d_k = 1$.

To lower the variance of a set of numbers, we divide all of them by the same number.

1. **In words:** scaling every value by c scales every distance from the mean by c, so the squared distances, and the variance, by c^2.
2. **Formula:** with $\mu = E[X]$,

$$\begin{aligned}\mathrm{Var}(cX) &= E\Big[(cX - c\mu)^2\Big] \\ &= c^2\, E\Big[(X - \mu)^2\Big] \\ &= c^2\, \mathrm{Var}(X)\end{aligned}$$

3. **Example:** the numbers $10, 20, 30, 40, 50, 60, 70$ have mean 40. Their squared distances from 40 are added in two halves, then averaged over the 7 numbers to give the variance:

$$30^2 + 20^2 + 10^2 + 0 = 1400$$

$$10^2 + 20^2 + 30^2 = 1400$$

$$\frac{1400 + 1400}{7} = \frac{2800}{7}$$

$$= 400$$

Divided by 10 they become $1, 2, \ldots, 7$. Their variance (Figure 7) is:

$$\frac{400}{10^2} = 4$$

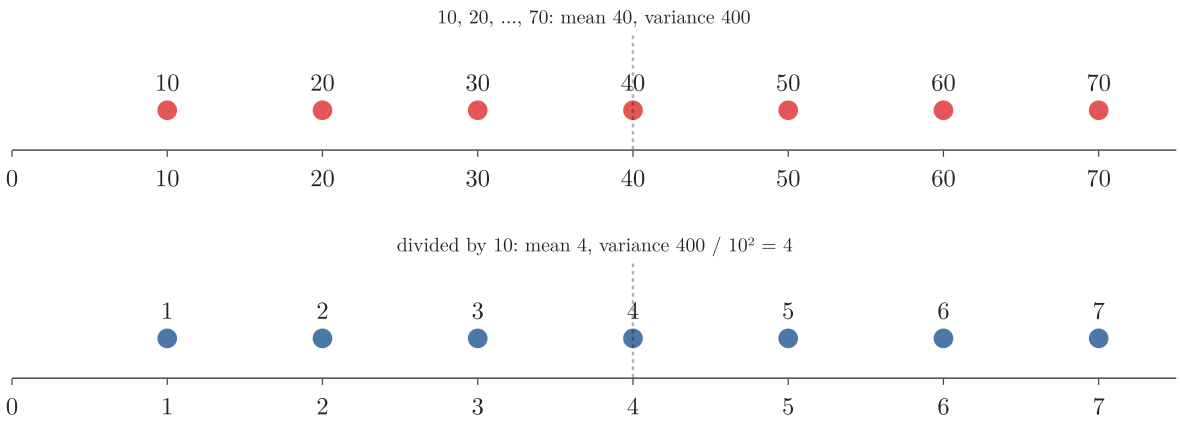


Figure 7: The worked example: every value divided by 10. The spread around the mean shrinks 10 times, so the variance shrinks $10^2 = 100$ times

Which c do we need? The scores have variance d_k (section 4.1), and we want variance 1, the same for every length:

$$\begin{aligned}
&\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) \\
&= \frac{1}{(\sqrt{d_k})^2}\,\mathrm{Var}(q \cdot k) \\
&= \frac{d_k}{d_k} \\
&= 1
\end{aligned}$$

So the **scaling factor** (G-1747) is:

$$c = \frac{1}{\sqrt{d_k}}$$

We divide by $\sqrt{2}$ when $d_k = 2$, by $\sqrt{3}$ when $d_k = 3$, and by 8 when $d_k = 64$. The Notebook confirms it: after scaling, the variance is 0.99, 0.98, 1.00, …, 1.02 for $d = 1$ to 1,024 (Figure 4, left), and the three histograms of Figure 3 (right) lie on top of each other.

Extra: Before the transformer, two kinds of **attention score** (G-223) were common: additive (a small neural network, Bahdanau's) and dot-product (Luong's), compared in the two attention scores side by side. Vaswani et al. (2017, section 3.2.1) report that the two perform similarly for small d_k, but additive attention does better than unscaled dot-product attention for larger d_k. Dot-product attention is "much faster and more space-efficient in practice", because it is one matrix multiplication. Scaling keeps that speed and removes the weakness.

7. The full formula in code

Key point: $\text{softmax}(QK^T/\sqrt{d_k})\,V$ computed by hand in NumPy gives exactly the weights and outputs of Keras' `MultiHeadAttention` layer with one head.

Keras implements **scaled dot-product attention** (G-1745) inside `keras.layers.MultiHeadAttention` (G-121). With `num_heads=1` it is a single attention: its weights are W_Q, W_K, W_V and one extra output matrix that **multi-head attention** (G-1268) needs. Setting the output matrix to the identity makes the layer compute exactly the formula of this Note.

Python: Scaled dot-product attention by hand, checked against Keras ("money bank grows", 8 numbers per word).

```
mha = keras.layers.MultiHeadAttention(num_heads=1, key_dim=8,
↪  use_bias=False)
mha(X, X)                                          # builds the layer
Wq, Wk, Wv, Wo = mha.get_weights()                 # shapes (8,1,8) x 3
↪  and (1,8,8)
mha.set_weights([Wq, Wk, Wv, np.eye(8)[None]])     # output matrix =
↪  identity
out_keras, w_keras = mha(X, X, return_attention_scores=True)

x = X[0]                                           # 3 words x 8
↪  numbers
Q, K, V = x @ Wq[:, 0], x @ Wk[:, 0], x @ Wv[:, 0]
A = softmax(Q @ K.T / np.sqrt(8))                  # the attention
↪  weights
Y = A @ V                                          # the contextual
↪  embeddings
```

The attention weights by hand (rows: "money", "bank", "grows" as queries) are

$$A = \begin{pmatrix} 0.015 & 0.670 & 0.316 \\ 0.159 & 0.575 & 0.267 \\ 0.250 & 0.116 & 0.634 \end{pmatrix}$$

and Keras returns the same matrix (Figure 8). Figure 8 draws A as a grid: each row is one word as the query, each column the word it looks at, and a darker cell means a larger weight.

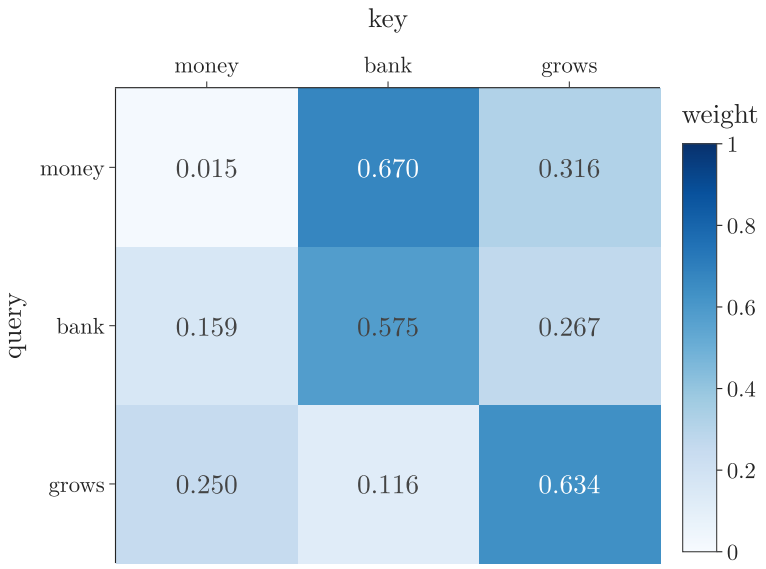


Figure 8: The attention weights of "money bank grows" from the hand computation; Keras' `MultiHeadAttention` with one head returns the same numbers. Each row sums to 1

The outputs agree too: the largest difference over the 3×8 numbers is 2×10^{-7}, the rounding error of 32-bit numbers (Notebook). The weights here are random, untrained values, so the pattern itself has no meaning yet; training would set W_Q, W_K and W_V.

8. Summary

Step	Formula	Why
Scores	QK^T	one dot product per (query, key) pair
Scale	divide by $\sqrt{d_k}$	the scores have variance d_k; scaling brings it to 1
Weights	softmax, row by row	each row of weights sums to 1
Output	weights $\times V$	contextual embeddings

- d_k is the length of the key (and query) vectors, so it sets how many terms each score adds up.
- The variance of a dot product of independent numbers with mean 0 and **variance** (G-2074) 1 equals the length of the vectors, because each of the d_k products adds its own unit of variance: 3, 100 and 1,000 gave measured variances 2.9, 96.7 and 1,015.6.
- Widely spread scores saturate the softmax: one weight near 1, a gradient near 0, so W_Q and W_K hardly learn. At $d = 1{,}024$ the largest weight averaged 0.97 and the gradient was 7.6 times smaller than with scaling.
- $\mathrm{Var}(cX) = c^2 \mathrm{Var}(X)$, so dividing by $\sqrt{d_k}$ makes the variance 1 for every d_k, which lets the vectors stay long without saturating the softmax.
- Keras' `MultiHeadAttention` with one head computes exactly $\mathrm{softmax}(QK^T/\sqrt{d_k})\,V$, so the hand formula is the one real libraries use.
- That answers both questions of the Note: d_k is the key length, and $\sqrt{d_k}$ is exactly the divisor that brings the score variance back to 1.

9. Sources

Built from

- CampusX, "Scaled Dot Product Attention | Why do we scale Self Attention?", YouTube, https://www.youtube.com/watch?v=r7mAt0iVqwo

Other references

- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Section 3.1 ($d_{model} = 512$); section 3.2.1 and eq. 1 (scaled dot-product attention, additive vs dot-product, small gradients); footnote 4 (variance d_k); section 3.2.2 ($d_k = 64$).
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026), chapter 7, section on attention, eq. 7.10–7.13.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Section 6.2.2.3 (the softmax saturates when the differences between its inputs become extreme).
- Maas, A. L. et al. (2011). Learning Word Vectors for Sentiment Analysis. *ACL 2011*. The IMDB review dataset (via `keras.datasets.imdb`).
- Keras API documentation: `MultiHeadAttention` layer, keras.io/api/layers/attention_layers/multi_he

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Scaled dot-product attention (G-1745)	Self-attention in which every query–key dot-product score is divided by $\sqrt{d_k}$ before the softmax, so the scores do not spread so wide that the softmax saturates: $\text{softmax}(QK^T/\sqrt{d_k})\,V$.
Saturated softmax (G-1740)	A softmax whose inputs are so far apart that one weight is near 1 and the rest near 0; its gradient is then near 0, so training barely changes the scores.
Softmax gradient (G-1831)	How much each softmax output changes when one input score changes: $\partial\alpha_i/\partial s_i = \alpha_i(1-\alpha_i)$ and $\partial\alpha_i/\partial s_j = -\alpha_i\alpha_j$; training changes the scores through it, and it is near 0 everywhere when the softmax is saturated.
Scaling factor (G-1747)	The number $1/\sqrt{d_k}$ that multiplies every attention score; it shrinks the spread of the scores so the softmax does not saturate.
`MultiHeadAttention` (G-121)	The Keras layer that computes multi-head attention: it learns W_Q, W_K, W_V and an output matrix and applies scaled dot-product attention; with `num_heads=1` it is a single attention.
d_k (G-24)	The length of each key vector (and query vector); attention divides its scores by $\sqrt{d_k}$ so that long vectors do not give scores so spread out that the softmax puts almost all the weight on one word.
Attention score (G-223)	A number that measures how strongly one word relates to another: the dot product of the first word's query with the second word's key. The softmax turns one word's scores into its attention weights.
Variance (of data) (G-2074)	The average squared distance of the values from their mean; the square of the standard deviation; divide by n for a population (NumPy default) or $n-1$ for a sample (pandas default).

Term	Meaning
Variance of a random variable (G-2076)	How spread out a random variable's values are: the expected squared distance from its expected value, $\mathrm{Var}(X) = E[(X - E[X])^2]$.
Vanishing gradient (G-2070)	Gradients shrinking towards 0 as they pass through many layers, which slows learning.
Multi-head attention (G-1268)	Several self-attentions (heads) run in parallel on the same input, each with its own weights so each can capture a different kind of relation between words; their outputs are joined and mixed by one more learned matrix, W_O.